YEAR 12 · AI ENGINEERING · LECTURE 4

Human in the Loop

Keeping people in charge of AI decisions

The AI does the work. A human makes the call when it matters.

TODAY

1. What HITL means
2. In, on, or out of the loop
3. HITL across the AI lifecycle
4. Benefits and challenges
5. Where to put the human
6. HITL in LangGraph: interrupt
7. Approve / edit / reject tools
8. Case studies + quiz

What is human-in-the-loop (HITL)?

- A way of designing AI systems where **people take part in the decisions**
- Humans can **teach** the AI (label data, give feedback), **check** it (review outputs) and **overrule** it
- The AI handles the **routine**, fast work; humans handle **uncertain, high-stakes or unusual** cases
- Goal: combine AI's **speed and scale** with human **judgement, context and accountability**
- Examples you know:
  - A teacher checking an AI-marked quiz before results go out
  - A bank calling you to confirm an unusual card payment
  - A driver taking over from a car's lane-assist

TALKING POINTS

Ask: 'Would you let an AI decide your final exam grade with no teacher checking?' Most will say no — then ask why. The reasons they give (mistakes, fairness, someone to appeal to) are exactly the reasons for HITL. The key phrase: AI for scale, humans for judgement.

In the loop, on the loop, out of the loop

How much control a person keeps depends on the **risk** of the decision

Level	What the human does	Example
Human-in-the-loop	Must **approve** each important decision before it happens	A doctor confirms an AI-suggested diagnosis
Human-on-the-loop	AI acts on its own; a human **monitors** and can step in or stop it	A moderator watches an auto-moderation dashboard
Human-out-of-the-loop	AI decides and acts with **no** human involvement	A spam filter moving junk emails

TALKING POINTS

Draw a slider from 'full human control' to 'full automation'. Ask students to place: autocorrect (out), a self-driving taxi with a remote operator (on), sentencing in court (in — and many argue AI shouldn't be involved at all). There's no single right level; it depends on harm.

HITL across the AI lifecycle

Humans can be part of **every** stage, not just the end



TALKING POINTS

Active learning: instead of labelling 100,000 random photos, the model asks a human to label the examples it's least sure about — much less human effort. RLHF (reinforcement learning from human feedback): people compare two answers and say which is better; that's how chatbots learned to be helpful. Today we focus on the deployment stage: approving an agent's actions.

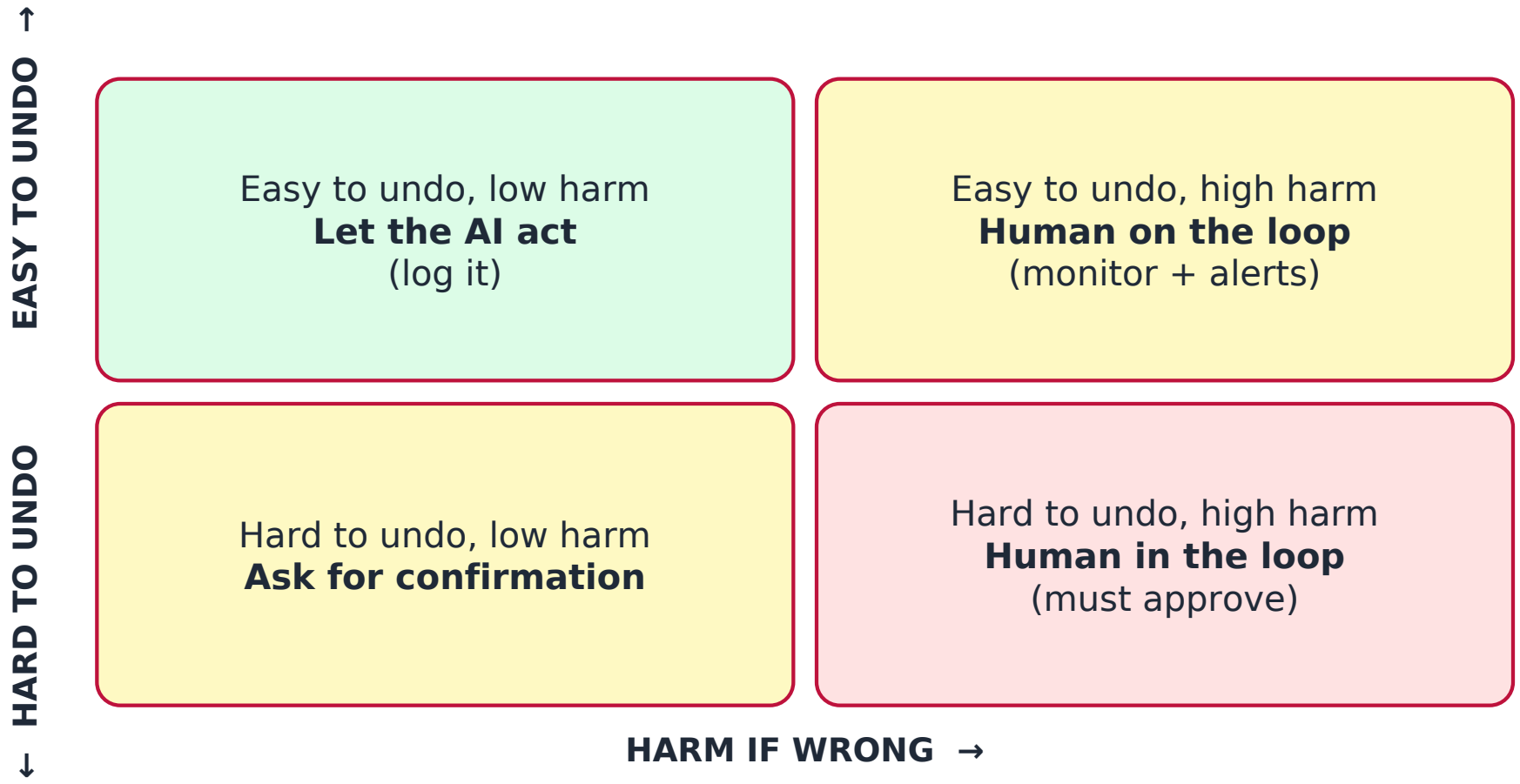
Benefits and challenges

Benefits ✓	Challenges ✗
Accuracy — humans catch mistakes and edge cases	**Speed** — waiting for a person slows things down
Accountability — a person is responsible for the decision	**Cost and scale** — people can't review millions of cases
Fairness — humans can spot unfair or biased outcomes	**Human bias** — reviewers have their own biases too
Trust — users accept AI more when people oversee it	**Automation bias** — people start clicking "approve" without reading
Learning — corrections improve the model over time	**Fatigue** — too many requests → careless reviews

TALKING POINTS

Spend time on automation bias: when an AI is usually right, reviewers stop checking properly and become a rubber stamp. Good HITL design asks for review only when it matters and shows the reviewer the evidence they need.

Where should the human go? Risk × reversibility



TALKING POINTS

Use the 2×2 grid to decide. Low-risk, easy-to-undo actions (looking something up) can be automatic. High-risk or irreversible ones (sending a message to all parents, deleting records, paying money) need approval. Ask students to place: 'search the handbook', 'email one student', 'post on the school Instagram', 'change a grade'.

HITL for AI agents

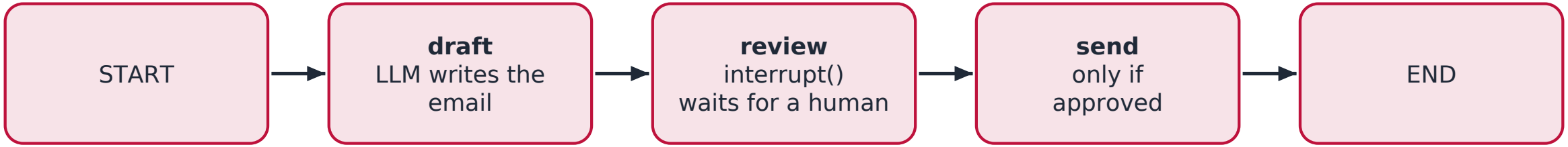
- Agents **act**: send emails, book things, change data, spend money
- Typical HITL points in an agent:
  - **Approve** a tool call before it runs ("Send this email to 200 parents?")
  - **Edit** the tool call (fix the date in the email), then run it
  - **Reject** it and tell the agent why
  - **Answer a question** the agent can't ("Which class is this for?")
  - **Review** the final output before it's published
- To pause for a human, the system must **save its state** and **resume later** — minutes or days later
- That's exactly what LangGraph's **checkpointers** + **interrupts** do

TALKING POINTS

Connect to Lecture 2: the checkpointer saves the state at every step, so the graph can stop, wait for a person (who might reply tomorrow), and continue exactly where it left off. Without saved state you'd have to start again from scratch.

interrupt(): pause the graph, ask a human, resume

Inside a node, `interrupt(info)` stops the graph and hands `info` to your app. `Command(resume=answer)` continues it.



1. `app.invoke(...)` runs until the interrupt and returns `__interrupt__` with the draft
2. Your app shows the draft to a person (web page, Slack, email…)
3. `app.invoke(Command(resume="yes"), config)` continues from the **same thread** — the value "yes" becomes the return value of `interrupt()`

TALKING POINTS

Emphasise that the graph really stops — no process needs to keep running while the teacher decides. The thread_id plus the checkpointer are how it finds its place again.

Human approval in LangGraph (1/2): the nodes


```
from typing import TypedDict
from langgraph.types import interrupt, Command

class EmailState(TypedDict):
    draft: str
    approved: bool

def draft(state):
    text = llm.invoke("Write a 2-sentence email to parents: "
                      "the science trip is on Friday.").content
    return {"draft": text}

def review(state):
    # pauses the whole graph and hands this info to the app
    decision = interrupt({"question": "Send this email?", "draft": state["draft"]})
    return {"approved": decision == "yes"}

def send(state):
    print("SENT" if state["approved"] else "NOT SENT")
    return {}
```


TALKING POINTS

The only new idea is interrupt() inside review. Whatever the human eventually sends back becomes the value of decision. Ask: where would you add an 'edit' option? (The human could return a corrected draft instead of 'yes'.)

Human approval in LangGraph (2/2): wiring + checkpointer


```
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver

g = StateGraph(EmailState)
g.add_node("draft", draft)
g.add_node("review", review)
g.add_node("send", send)
g.add_edge(START, "draft")
g.add_edge("draft", "review")
g.add_edge("review", "send")
g.add_edge("send", END)

# a checkpointer is needed to pause + resume
app = g.compile(checkpointer=InMemorySaver())
```

Three essentials for HITL:

1. `interrupt()` in a node
2. a **checkpointer** when compiling
3. a `thread_id` when running (next slide)

TALKING POINTS

Without the checkpointer, LangGraph has nowhere to save the paused state, so interrupt() can't work. In a real app the checkpointer would be a database so the pause can survive a server restart.

Running it: pause → human decides → resume

The first call stops at the interrupt; the second call continues from the saved checkpoint

```
config = {"configurable": {"thread_id": "trip-email-1"}}

result = app.invoke({"draft": "", "approved": False}, config)
print(result["__interrupt__"][0].value)
# {'question': 'Send this email?', 'draft': 'Dear parents, ...'}

# ... a teacher reads the draft (seconds, hours or days later) ...

answer = input("Approve? (yes/no) ")
# resume on the SAME thread_id:
final = app.invoke(Command(resume=answer), config)
print(final)
```

Try it:

- Answer *no* — the email isn't sent
- Change `review` so the human can type a **corrected** draft
- Add a loop: if rejected, go back to `draft` with the human's feedback (max 2 times)

TALKING POINTS

If you can, run this live in a notebook. The moment the cell stops and waits for the teacher's 'yes' makes the idea click. Then run it again and answer 'no'.

Shortcut: approve / edit / reject tool calls in a LangChain agent


```
from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command

agent = create_agent(
    llm, tools=[search_handbook, send_email],
    middleware=[HumanInTheLoopMiddleware(interrupt_on={
        "send_email": {"allowed_decisions": ["approve", "edit", "reject"]},
        "search_handbook": False,                    # safe: no approval needed
    })],
    checkpointer=InMemorySaver())

cfg = {"configurable": {"thread_id": "office-1"}}
out = agent.invoke({"messages": [{"role": "user",
      "content": "Email all Year 12 parents that the trip moved to Monday."}]}, cfg)
# out["__interrupt__"] shows the proposed send_email call -> a human decides:
agent.invoke(Command(resume={"decisions": [{"type": "approve"}]}), cfg)
```


TALKING POINTS

This is the risk × reversibility grid in code: searching is safe so it runs automatically; sending email is hard to undo, so it needs approval, and the human may also edit (e.g. fix the date) or reject. Under the hood it's the same interrupt mechanism.

Case studies — what would you do?

Scenario	Your HITL design?
An AI marks Year 12 practice essays and writes feedback	
A chatbot answers parents' questions about school rules (RAG)	
An agent can change students' timetables	
An AI flags possible cyberbullying in school chat channels	
A hospital AI suggests which patients to see first in the emergency department	

TALKING POINTS

Pairs choose: in / on / out of the loop, and which actions need approve / edit / reject. Suggested answers: essays — on the loop with teacher spot-checks and an appeal route; parent chatbot — out of the loop for answers with citations, human handover for complaints; timetable changes — in the loop (approval); cyberbullying — AI flags, a trained human decides (never automatic punishment); hospital triage — in the loop, clinicians decide, AI assists. Discuss automation bias for the hospital case.

Recap + exit quiz

- **HITL** = people teach, check and overrule AI where it matters
- **In** the loop (approve) · **on** the loop (monitor) · **out** of the loop (automatic) — choose by **risk × reversibility**
- Watch for **automation bias** and reviewer **fatigue**
- LangGraph: `interrupt()` + checkpointer + `Command(resume=…)`; agents: `HumanInTheLoopMiddleware`
- **Quiz:** 1) Difference between in and on the loop? 2) Why does pausing need a checkpointer? 3) Give one action an agent should never take without approval. 4) What is automation bias?

TALKING POINTS

Answers: 1) In = must approve before the action; on = AI acts, human monitors and can intervene. 2) The state must be saved so the graph can resume later from the same point. 3) Any hard-to-undo, high-harm action: sending mass emails, payments, deleting data, changing grades. 4) Trusting the AI too much and approving without really checking. Wrap up the four-lecture series: LangChain (parts) → LangGraph (wiring) → RAG (knowledge) → HITL (human control).